



Figure 3: Kernel configuration – making changes

```
#Set the path for Android build env (64 bit)
export PATH=${HOME}/android/ndk/android-ndk-r9/toolchains/
arm-linux-androideabi-4.4.3/prebuilt/linux-x86_64/
bin:$PATH
```

Step 4: Configure the Android kernel

Install the necessary dependencies, as follows:

```
$ sudo apt-get install libcurses5-dev build-essential
```

Set up the architecture and cross compiler, as follows:

```
$ export ARCH=arm
$ export CROSS_COMPILE=arm-linux-androideabi-
```

The kernel Makefile refers to the above variables to select the architecture and cross compile. The cross compiler command will be `$(CROSS_COMPILE)gcc` which is expanded to `arm-linux-androideabi-gcc`. The same applies for other tools like `g++`, `as`, `objdump`, `gdb`, etc.

Configure the kernel for the device:

```
$ cd ~/android/kernel
$ make mint-v1x-rev03_defconfig
```

The device-specific configuration files for ARM architecture are available in the `arch/arm/configs` directory.

Executing the configuration command may throw a few warnings. You can ignore these warnings now. The command will create a `.config` file, which contains the kernel configuration for the device.

To view and edit the kernel configuration, run the following command:

```
$ make menuconfig
```

Next, let's assume you want to change `lcd overlay` support.

Navigate to `Drivers` → `Graphics` → `Support for framebuffer devices`. The option to support `lcd overlay` should be displayed as shown in Figure 3.

Skip the `menuconfig` step or do not make any changes if

you are unsure.

Step 5: Build the kernel

Finally, we are ready to fire the build. Run the `make` command, as follows:

```
$ make zImage
```

If you want to speed up the build, specify the `-j` option to the `make` command. For example, if you have four processor cores, you can specify the `-j4` option to make:

```
$ make -j4 zImage
```

The compilation process will take time to complete, based on the options available in the kernel configuration (`.config`) and the performance of the build system. On completion, the kernel image (`zImage`) will be generated in the `arch/arm/boot/` directory of the kernel source.

Compile the modules:

```
$ make modules
```

This will trigger the build for kernel modules, and `.ko` files should be generated in the corresponding module directories. Run the `find` command to get a list of `.ko` files in the kernel directory:

```
$ find . -name "*.ko"
```

What next?

Now that you have set up the Android build environment, and compiled an Android kernel and necessary modules, how do you flash it to the handset so that you can see the kernel working? This requires the handset to be rooted first, followed by flashing the kernel and related software. It turns out that there are many new concepts to understand before we get into this. So be sure to follow the next article on rooting and flashing your custom Android kernel. 

References

<https://source.android.com/>
<https://developer.android.com/>
<http://xda-university.com>

By: Mubeen Jukaku

Mubeen is technology head at Emertxe Information Technologies (<http://www.emertxe.com>). His area of expertise is the architecture and design of Linux-based embedded systems. He has vast experience in kernel internals, device drivers and application porting, and is passionate about leveraging the power of open source for building innovative products and solutions. He can be reached at mubeen@emertxe.com